

```

        else:
            # Display the wall or empty space:
            print(maze[(x, y)], end='')
        print() # Print a newline after printing the row.

def visit(x, y):
    """Carve out" empty spaces in the maze at x, y and then
    recursively move to neighboring unvisited spaces. This
    function backtracks when the mark has reached a dead end."""
    maze[(x, y)] = EMPTY # "Carve out" the space at x, y.
    printMaze(maze, x, y) # Display the maze as we generate it.
    print('\n\n')

while True:
    # Check which neighboring spaces adjacent to
    # the mark have not been visited already:
    unvisitedNeighbors = []
    if y > 1 and (x, y - 2) not in hasVisited:
        unvisitedNeighbors.append(NORTH)

    if y < HEIGHT - 2 and (x, y + 2) not in hasVisited:
        unvisitedNeighbors.append(SOUTH)

    if x > 1 and (x - 2, y) not in hasVisited:
        unvisitedNeighbors.append(WEST)

    if x < WIDTH - 2 and (x + 2, y) not in hasVisited:
        unvisitedNeighbors.append(EAST)

    if len(unvisitedNeighbors) == 0:
        # BASE CASE
        # All neighboring spaces have been visited, so this is a
        # dead end. Backtrack to an earlier space:
        return
    else:
        # RECURSIVE CASE
        # Randomly pick an unvisited neighbor to visit:
        nextIntersection = random.choice(unvisitedNeighbors)

        # Move the mark to an unvisited neighboring space:

        if nextIntersection == NORTH:
            nextX = x
            nextY = y - 2

```